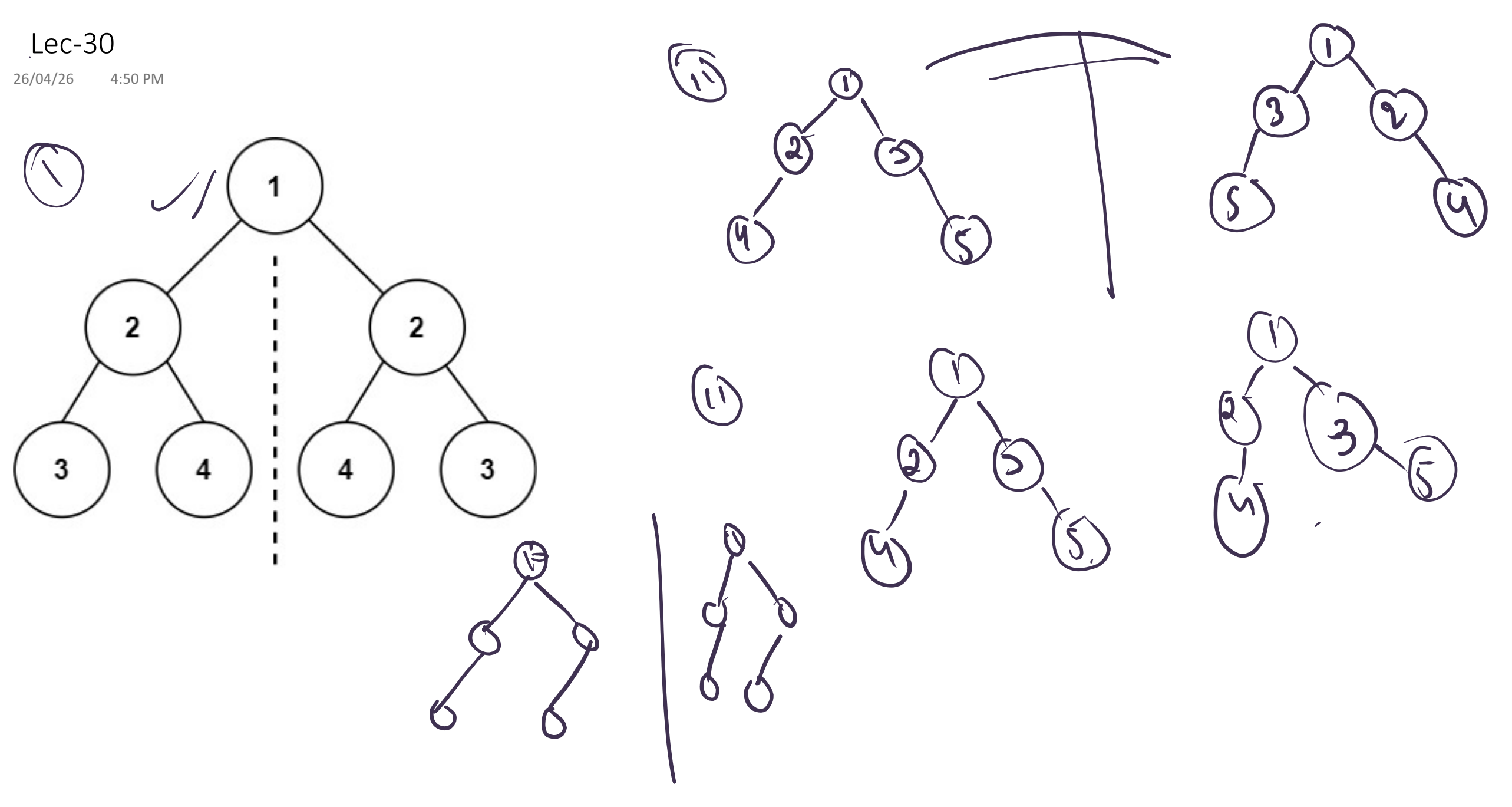
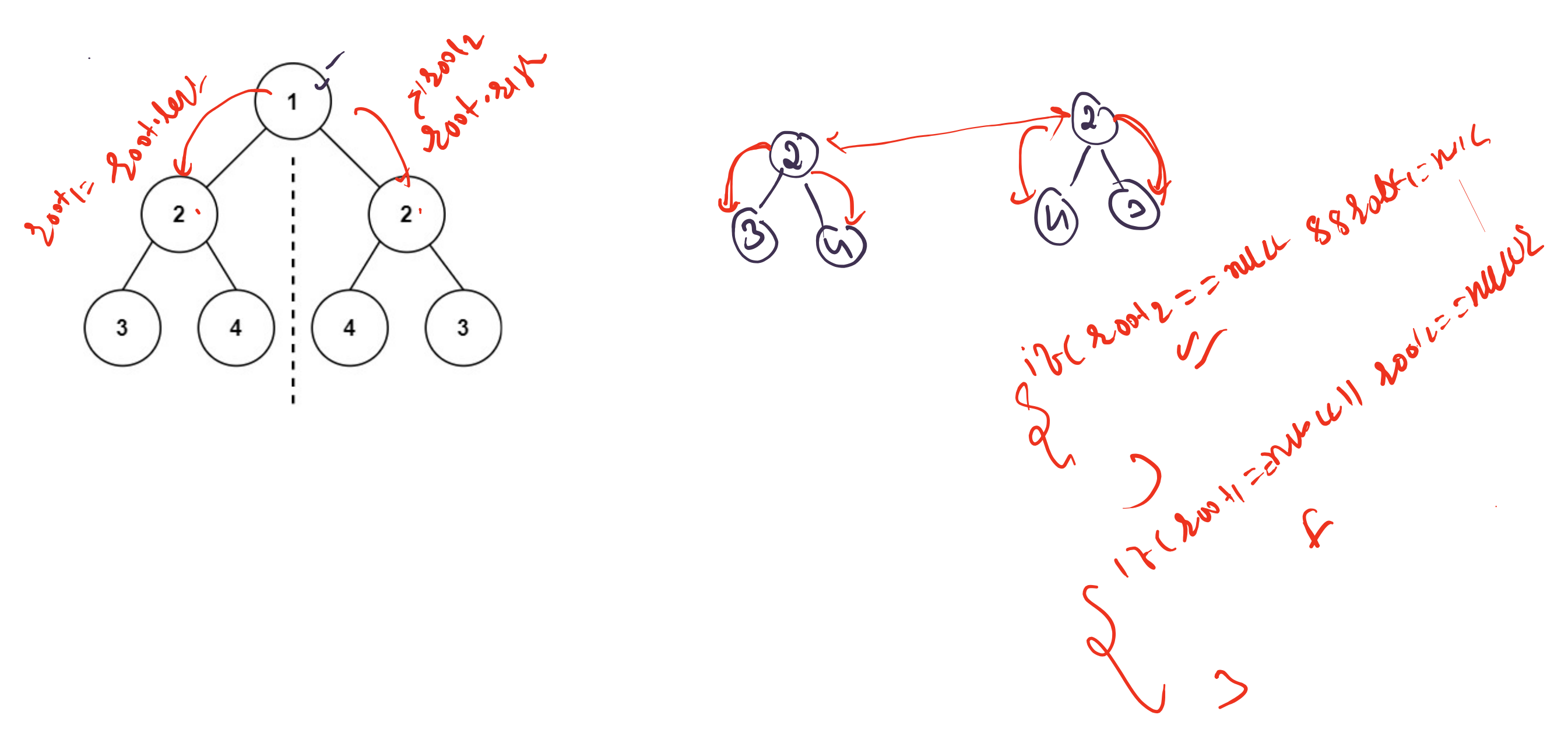
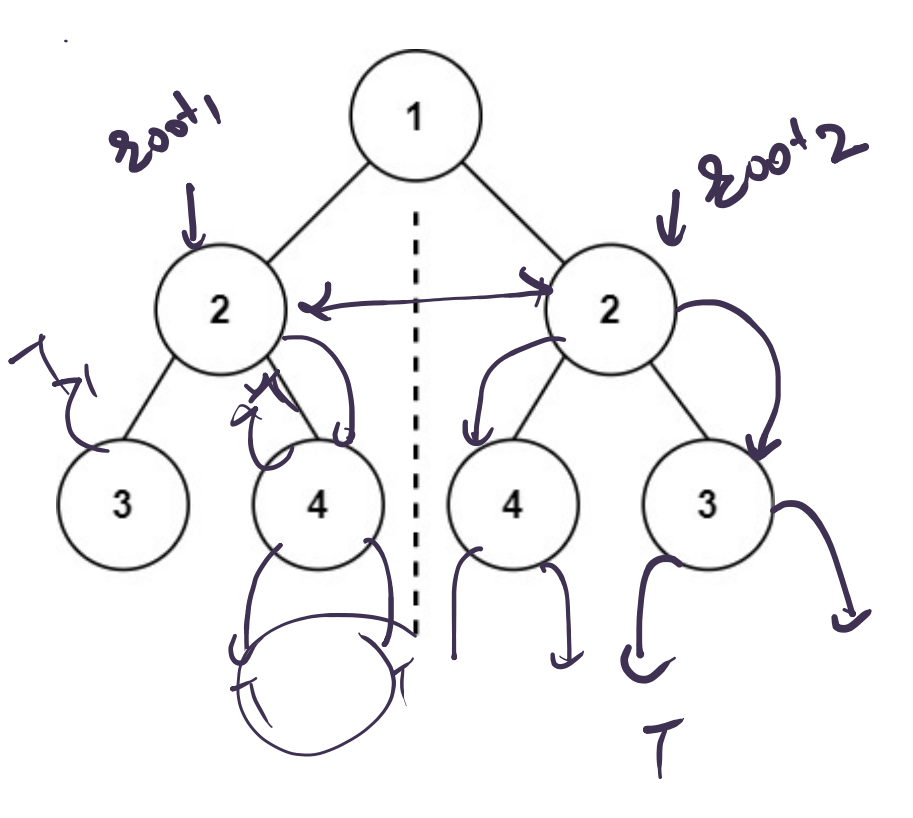
Lec-30

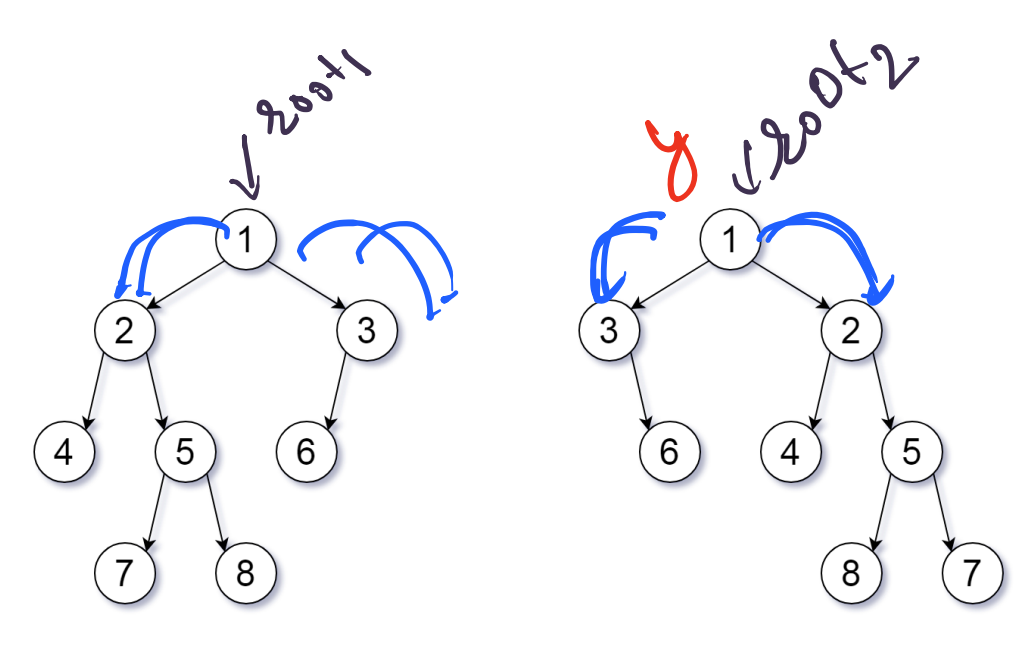
26/04/26 4:50 PM

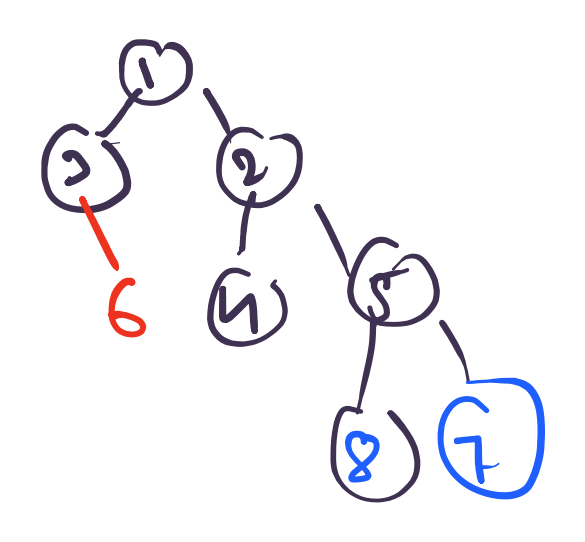


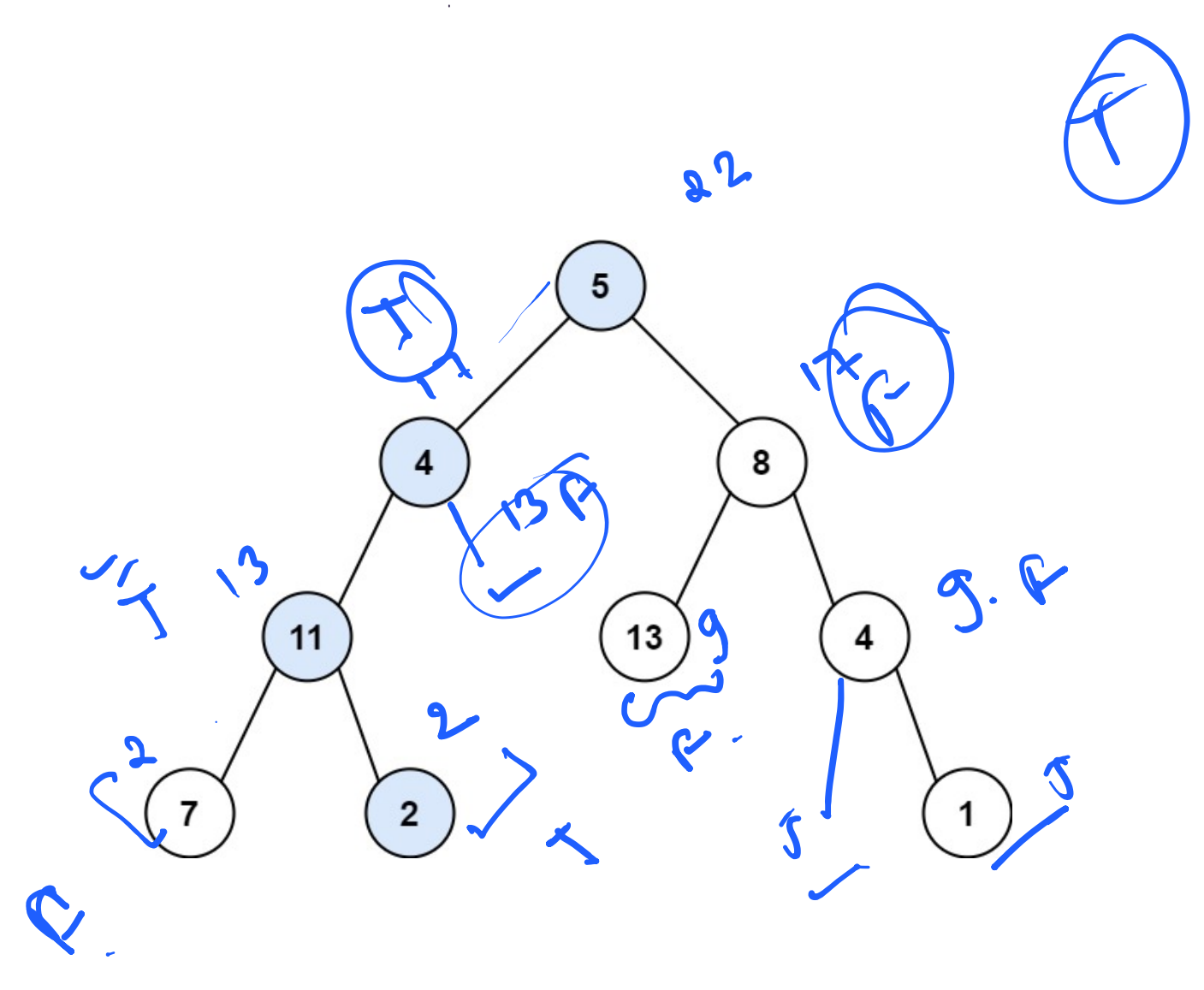

root1= root.left
root2= root.right
if(root2==null && root1==null)
{
}
if(root1==null|| root2==null)
{ f
}

```
public boolean ismirror(TreeNode root1,TreeNode root2) {
        if(root1==null && root2==null) {
            return true;
        }
        if(root1==null || root2==null) {
            return false;
        }
        if(root1.val!=root2.val) {
            return false;
        }
        boolean left=ismirror(root1.left, root2.right);
        boolean right=ismirror(root1.right, root2.left);
        return left && right;

    }
```


root1
root2
T

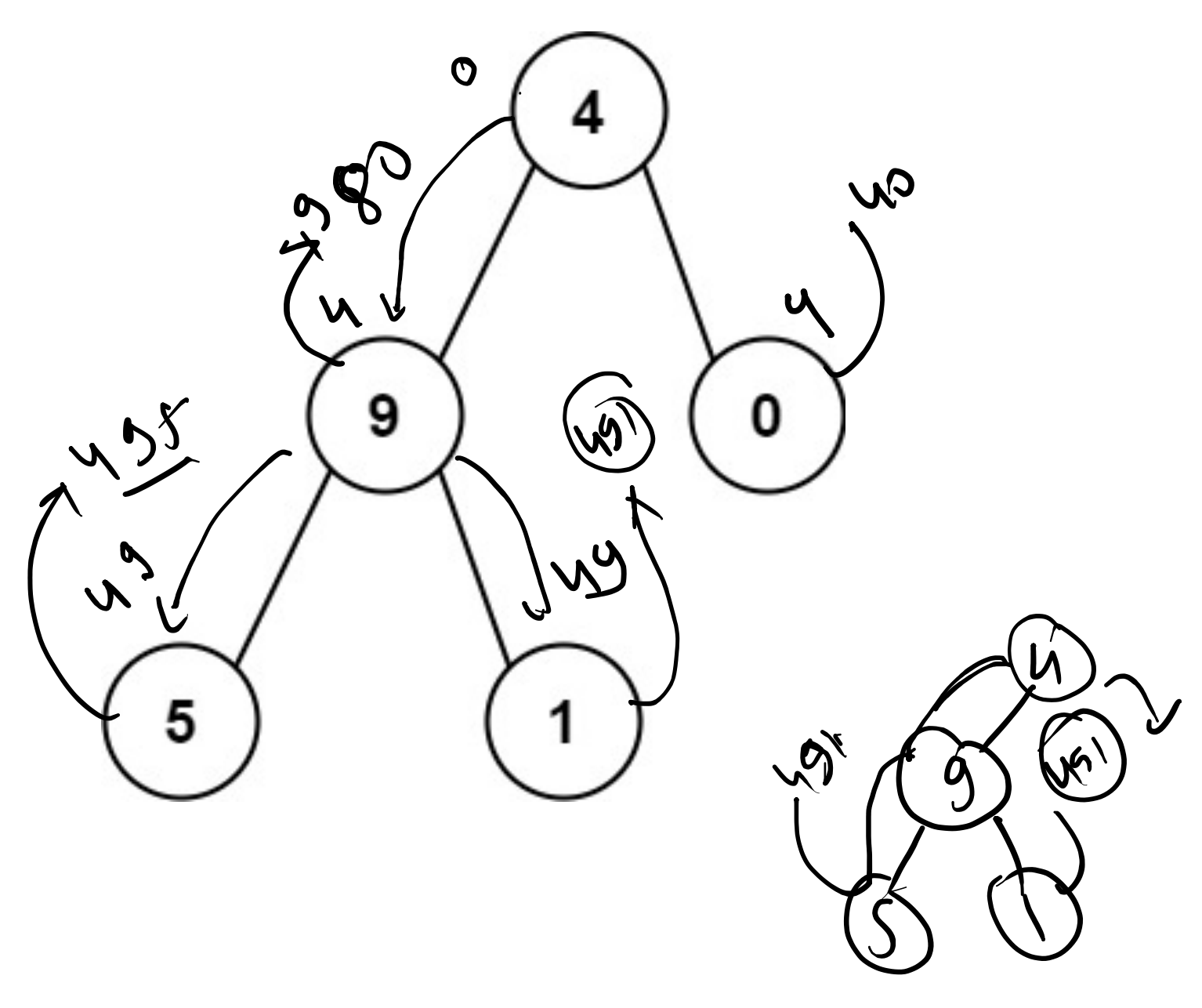

root1
root2




22
T
F
13 F
17 F
13
9 F
2
T
F

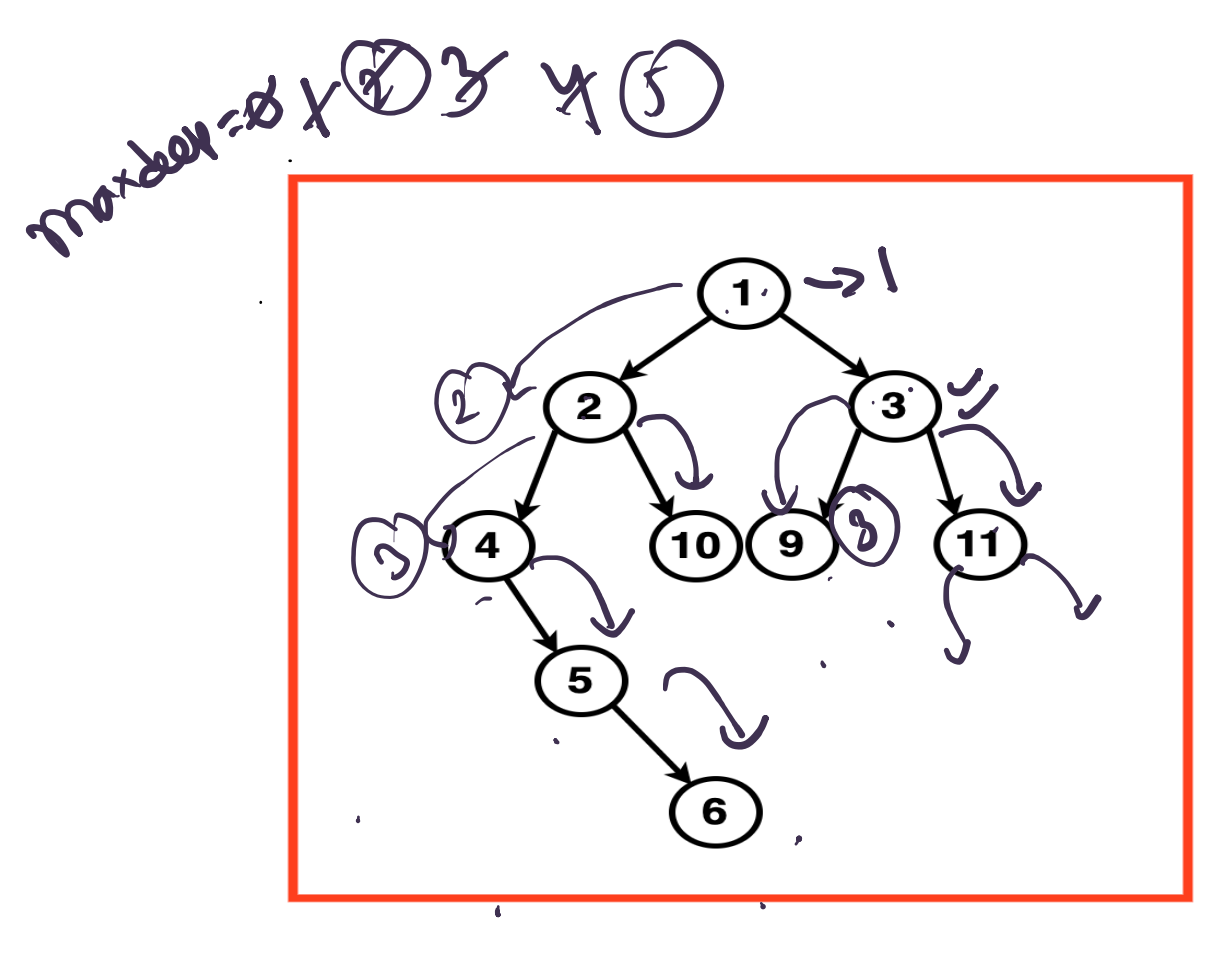
self t-root.val

sum


4
9
0
5
1
0
4980
40
495
491

495+491+40
= 1086

495+451
= 986+40
= 1026

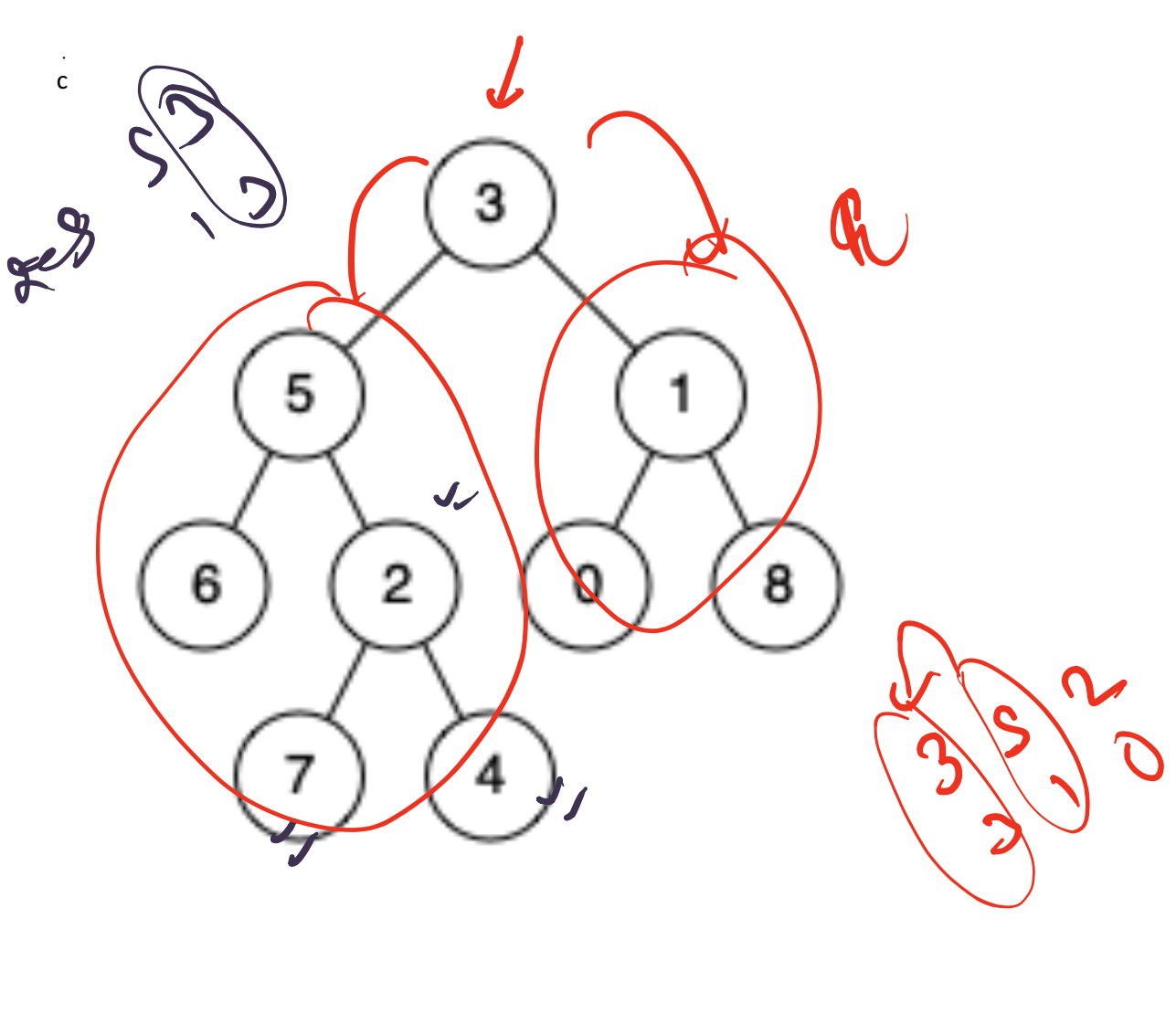

maxdepth=0 1 2 3 4 5

1 3 11 5 6

9

[1, 3, 11, 15, 6]

1
2 3
4 10 9 11
5
6

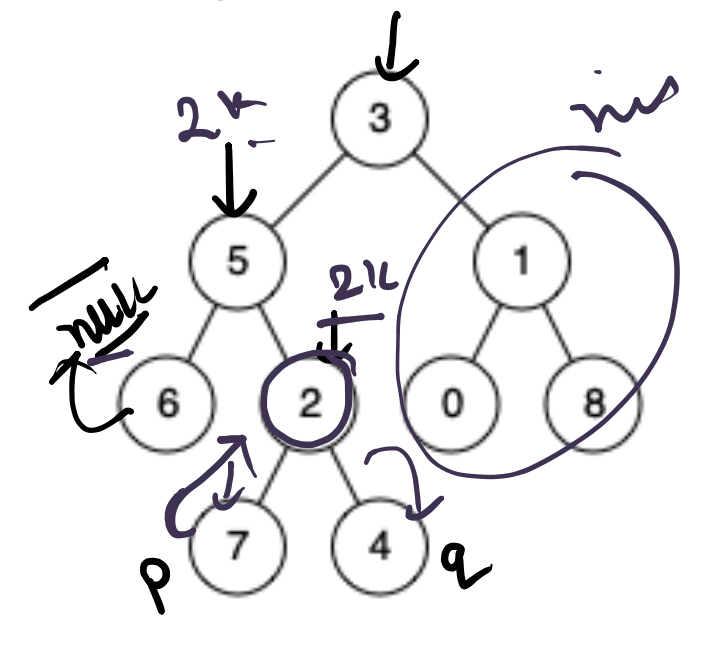


p=7 q=4

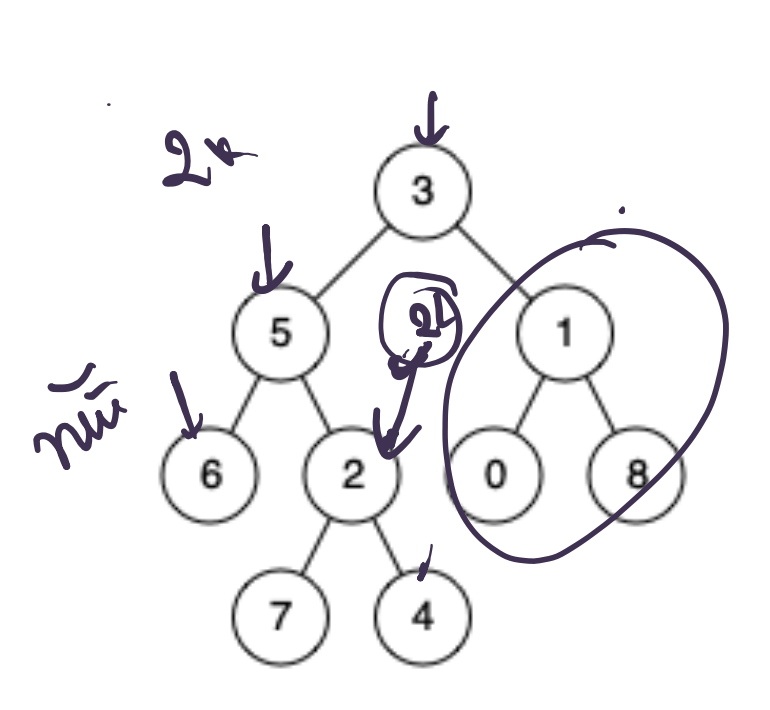
2 5 2
3 5 2 7
3 5 2 4

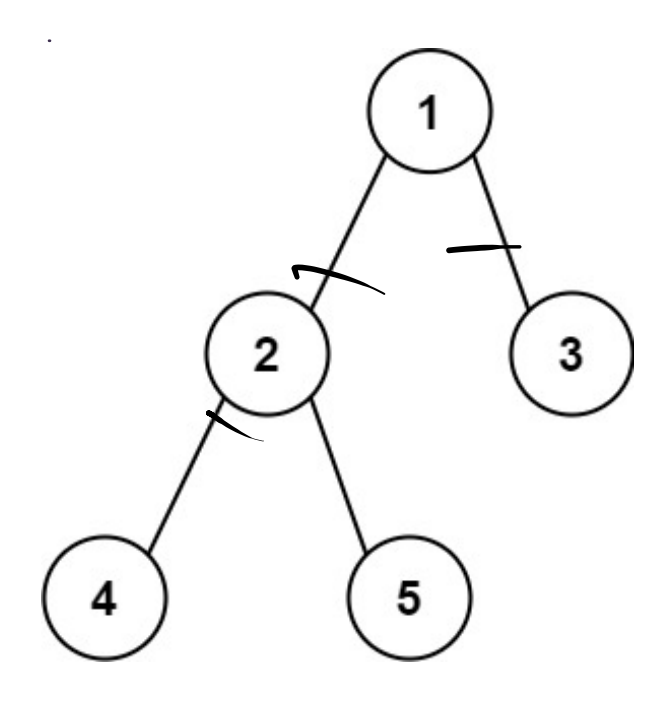
2 5 3
2 5 8
3 5 2
3 3 2

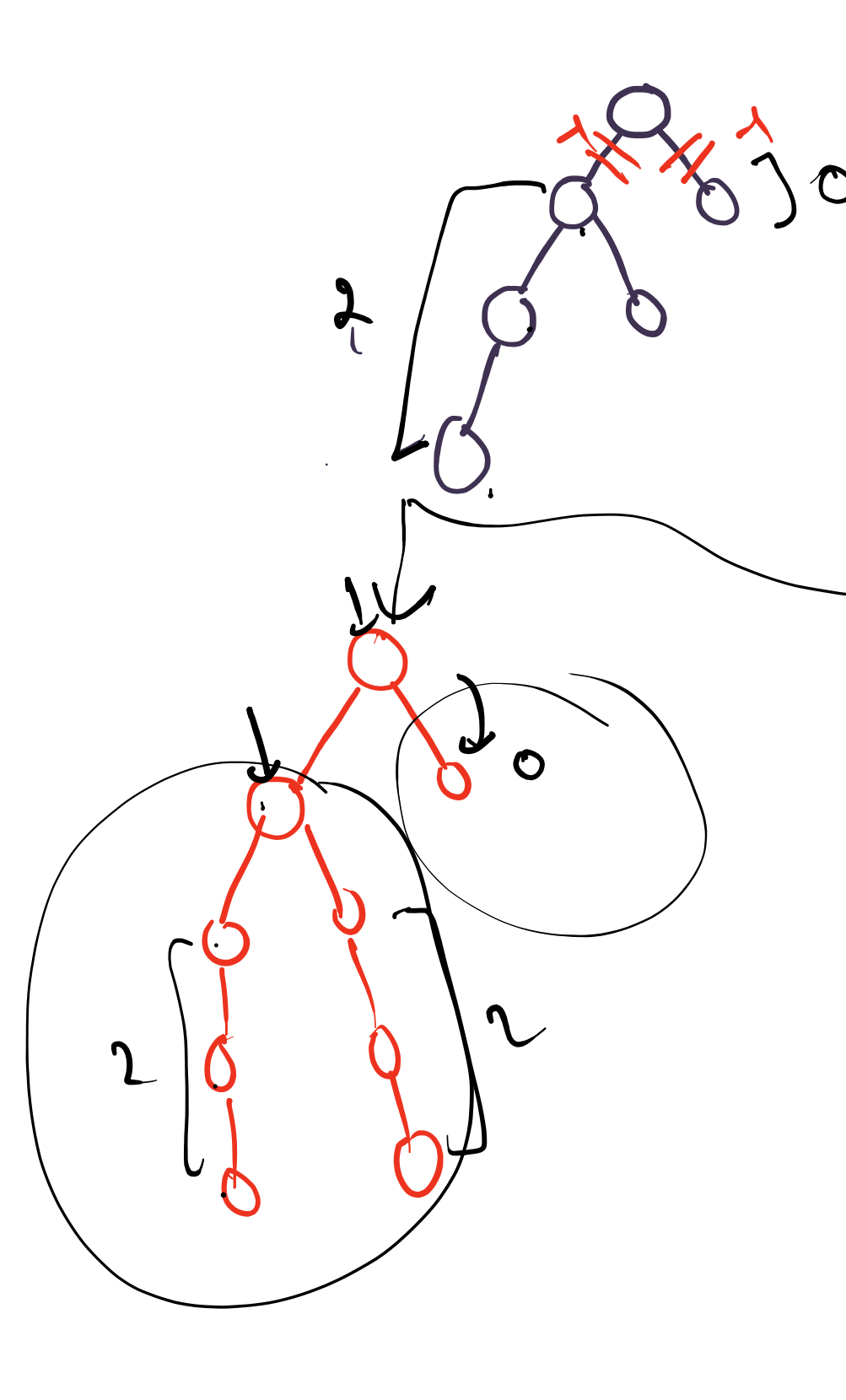
```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

    }
```



```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null) {
            return null;
        }
        if (root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if(left!=null && right!=null) {
            return root;
        }
        else if(left==null) {
            return right;
        }
        else {
            return left;
        }

    }
```







θ- maximum distance b/w any two nodes

2+0+2

lh+Rh+2

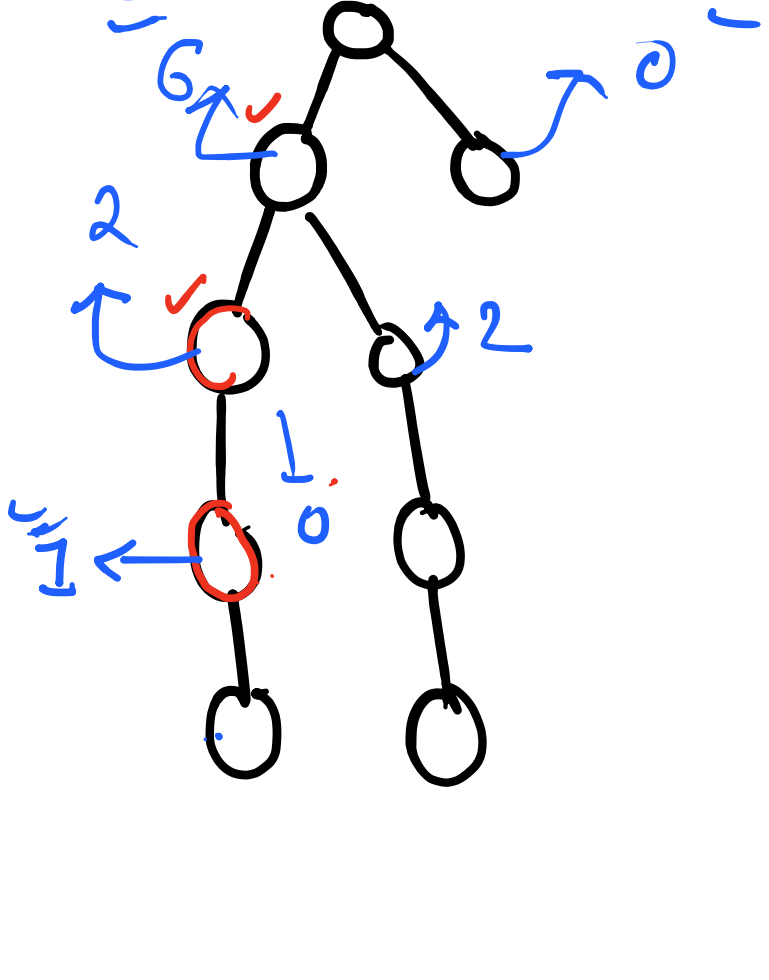
3+0+2=5

2+2+2=6

$T(n) = T(n/2) + T(n/2) + T(n/2) + T(n/2) + 1$

$= 4T(n/2)+1$

```
public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int ld = diameterOfBinaryTree(root.left);
        int rd = diameterOfBinaryTree(root.right);
        int sd = ht(root.left) + ht(root.right) + 2;
        return Math.max(ld, Math.max(sd, rd));
    }
```



0 -1+2=1

1 -1+2=2

ld=0

rd=0

-1-1+2=0

```
int ld = diameterOfBinaryTree(root.left);
int rd = diameterOfBinaryTree(root.right);
int sd = ht(root.left) + ht(root.right) + 2;
return Math.max(ld, Math.max(sd, rd));
```

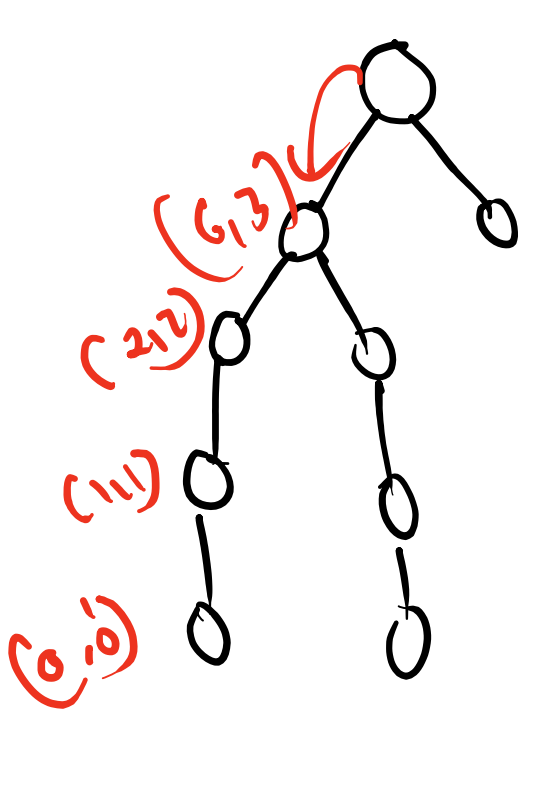
ldp

rdp

ldp.ht

rdp.ht

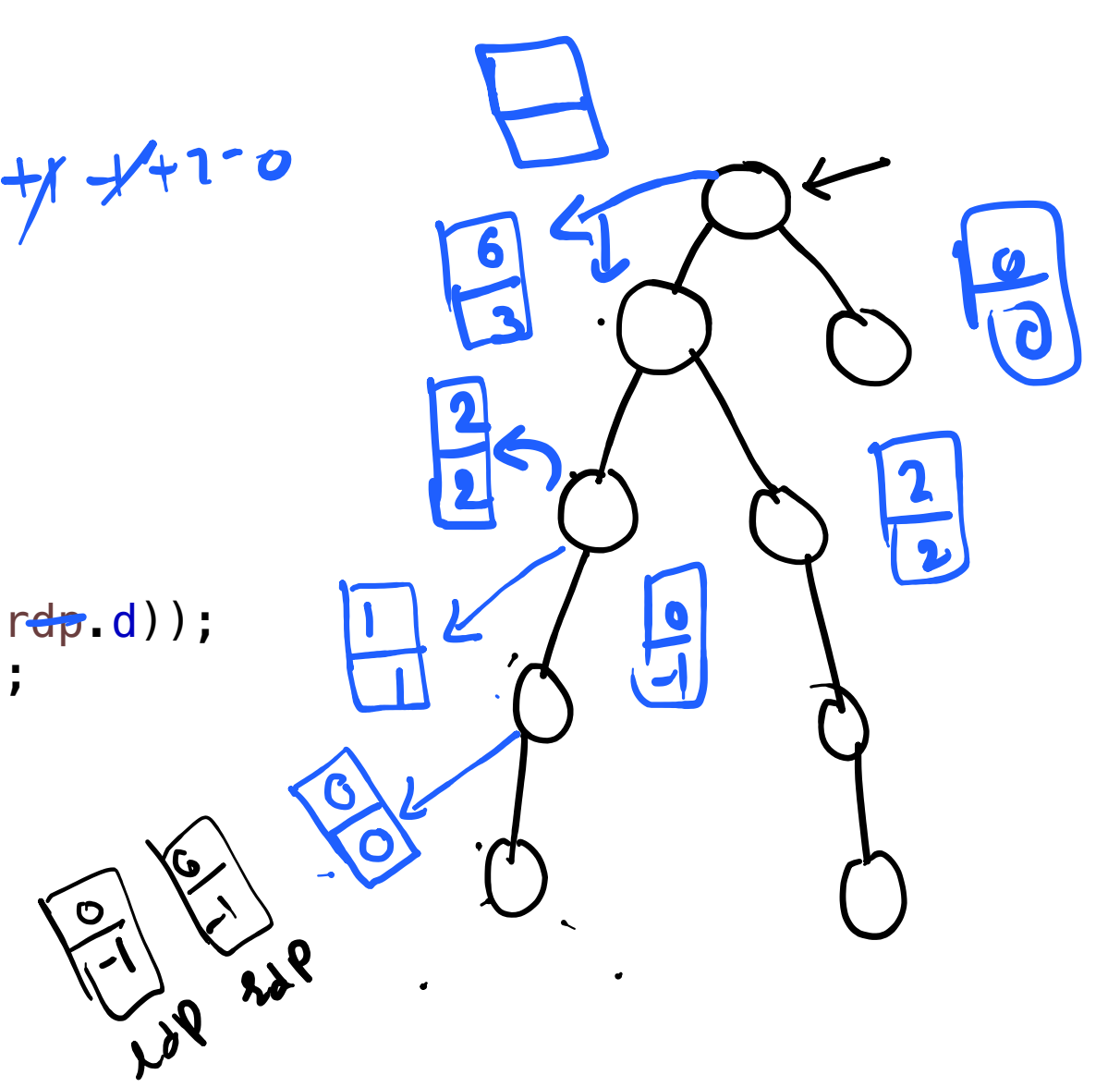
(ldp.ht, rdp.ht)+1



```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }
        DiaPair ldp = diameter(root.left);
        DiaPair rdp = diameter(root.right);
        int sd = ldp.ht + rdp.ht + 2;
        DiaPair sdp = new DiaPair();
        sdp.d = Math.max(ldp.d, Math.max(sd, rdp.d));
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        return sdp;
    }
```

0 -1+2=1

-1 -1+2=0


6
3
6
0
2
2
2
2
1
1
0
-1
0
0
0
-1
0
-1
ldp
rdp